Lahore University of Management Sciences
Department of Electrical Engineering

AI600 — Deep Learning

Spring 2026

Programming Assignment 3

Convolutional Netural Networks

GitHub Repository: https://github.com/uazam1163-debug/ai600-pa3
Date Submitted: 6th April, 2026

Contents

1 Analytical Questions **3**

2 Q1: Forward and Backward Pass **3**
2.1 Part 1a: Forward Pass . . . 3
2.2 Part 1b: Backward Pass . . . 3
2.3 Part 1c: Interpretation . . . 4

3 Q2: Image Perturbation & Receptive Fields [Practice — No Submission] **5**
3.1 Part 2a: $\partial\mathcal{L}/\partial X_{1,1}$ (centre pixel, 0-indexed) . . . 5
3.2 Part 2b: Adversarial Perturbation . . . 5
3.3 Part 2c: Weight Sharing and Compounding Gradient . . . 5

4 Q3: Pooling and Normalization **5**
4.1 Part 3a: Global Max Pooling . . . 5
4.2 Part 3b: Layer Normalization ($\gamma = 1$, $\beta = 0$, $\varepsilon = 0$) . . . 5
4.3 Part 3c: Translation Invariance . . . 6

5 Q4: Residual Networks [Practice — No Submission] **6**
5.1 Part 4a: Forward Pass . . . 6
5.2 Part 4b: Backward Pass . . . 6
5.3 Part 4c: Gradient Highway . . . 6

6 Programming Tasks **7**

7 Task 1A: Custom CNN on MNIST **7**
7.1 Architecture Design . . . 7
7.2 Training . . . 7
7.3 Results . . . 8
7.4 Analytical Question 1.1 — Generalisation Analysis . . . 8
7.5 Analytical Question 1.2 — First-Layer Filter Visualisation . . . 9

8 Task 1B: Colored-MNIST & Shortcut Learning **10**
8.1 Dataset & Adaptation . . . 10
8.2 Training Curves . . . 10
8.3 Results . . . 11
8.4 Analytical Question 1.3 — Why the Performance Drop? . . . 11
8.5 Analytical Question 1.4 — Debiasing Strategies . . . 11

9 Task 2A: Transfer Learning with ResNet-18 on STL-10 **12**
9.1 Model Setup . . . 12
9.2 Training Dynamics (Figure 7) . . . 13
9.3 Analytical Question 2.1 — Why Freeze Early Layers? . . . 13

10 Task 2B: GradCAM Visualisation **14**
10.1 Implementation . . . 14
10.2 Analytical Question 2.2 — Correct Predictions . . . 15
10.3 Analytical Question 2.3 — Incorrect Predictions . . . 15

11 AI Policy Compliance **16**

1 Analytical Questions

Model & Data Setup

The Toy-CNN processes a 3×3 grayscale input through one convolutional layer (2×2 filter, stride 1, no padding), ReLU, flatten, and a fully-connected layer. All computations use:

$$X = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{pmatrix}, \quad W = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}, \quad b = 0, \quad V = \begin{bmatrix} 1 & -1 & 2 & 0.5 \end{bmatrix}, \quad c = 1, \quad T = 3.$$

$$\mathcal{L} = \tfrac{1}{2}(Y - T)^2.$$

2 Q1: Forward and Backward Pass

2.1 Part 1a: Forward Pass

Step 1 — Convolution: $Z[i,j] = \sum_{k,l} X[i{+}k, j{+}l] \cdot W[k,l] + b$

$$\begin{aligned}
Z[0,0] &= (1)(1) + (2)(0) + (0)(-1) + (1)(1) = 1 + 0 + 0 + 1 = \mathbf{2} \\
Z[0,1] &= (2)(1) + (0)(0) + (1)(-1) + (1)(1) = 2 + 0 - 1 + 1 = \mathbf{2} \\
Z[1,0] &= (0)(1) + (1)(0) + (1)(-1) + (0)(1) = 0 + 0 - 1 + 0 = \mathbf{-1} \\
Z[1,1] &= (1)(1) + (1)(0) + (0)(-1) + (2)(1) = 1 + 0 + 0 + 2 = \mathbf{3}
\end{aligned}$$

$$Z = \begin{pmatrix} 2 & 2 \\ -1 & 3 \end{pmatrix}$$

Step 2 — ReLU activation: $A = \max(0, Z)$

$$A = \begin{pmatrix} 2 & 2 \\ \boxed{0} & 3 \end{pmatrix} \quad \longleftarrow \quad A[1,0] = \max(0,-1) = 0 \text{ (dead neuron; gradient blocked)}$$

Step 3 — Flatten and Fully Connected Layer

$$A_{\text{flat}} = \begin{bmatrix} 2 & 2 & 0 & 3 \end{bmatrix}^\top$$

$$Y = V \cdot A_{\text{flat}} + c = (1)(2) + (-1)(2) + (2)(0) + (0.5)(3) + 1 = 2 - 2 + 0 + 1.5 + 1 = \mathbf{2.5}$$

Step 4 — Loss

$$\mathcal{L} = \tfrac{1}{2}(Y - T)^2 = \tfrac{1}{2}(2.5 - 3)^2 = \tfrac{1}{2}(0.25) = \mathbf{0.125}$$

$Z = [[2,2],[-1,3]] \quad A = [[2,2],[0,3]] \quad Y = 2.5 \quad \mathcal{L} = 0.125$

2.2 Part 1b: Backward Pass

Gateway: $\partial\mathcal{L}/\partial Y$

$$\frac{\partial \mathcal{L}}{\partial Y} = Y - T = 2.5 - 3 = -0.5$$

Gradient w.r.t. FC weights V

Since $Y = V \cdot A_{\text{flat}} + c$:

$$\frac{\partial \mathcal{L}}{\partial V} = \frac{\partial \mathcal{L}}{\partial Y} \cdot A_{\text{flat}} = -0.5 \times [2,\ 2,\ 0,\ 3] = [-1.0,\ -1.0,\ 0.0,\ -1.5]$$

$\partial \mathcal{L}/\partial V = [-1.0,\ -1.0,\ 0.0,\ -1.5]$

Gradient back through FC to A

$$\frac{\partial \mathcal{L}}{\partial A_{\text{flat}}} = \frac{\partial \mathcal{L}}{\partial Y} \cdot V = -0.5 \times [1, -1, 2, 0.5] = [-0.5,\ 0.5,\ -1.0,\ -0.25]$$

Reshaped to 2×2 (row-major):

$$\frac{\partial \mathcal{L}}{\partial A} = \begin{pmatrix} -0.5 & 0.5 \\ -1.0 & -0.25 \end{pmatrix}$$

Through ReLU — masking dead neuron

$$\frac{\partial \mathcal{L}}{\partial Z} = \frac{\partial \mathcal{L}}{\partial A} \odot \mathbf{1}[Z > 0] = \begin{pmatrix} -0.5 & 0.5 \\ -1.0 & -0.25 \end{pmatrix} \odot \begin{pmatrix} 1 & 1 \\ \boxed{0} & 1 \end{pmatrix} = \begin{pmatrix} -0.5 & 0.5 \\ \boxed{0} & -0.25 \end{pmatrix}$$

Gradient w.r.t. convolutional filter W

$$\frac{\partial \mathcal{L}}{\partial W[k,l]} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial Z[i,j]} \cdot X[i+k,\ j+l]$$

$$\frac{\partial \mathcal{L}}{\partial W[0,0]} = (-0.5)(1) + (0.5)(2) + (0)(0) + (-0.25)(1) = -0.5 + 1.0 + 0 - 0.25 = +0.25$$
$$\frac{\partial \mathcal{L}}{\partial W[0,1]} = (-0.5)(2) + (0.5)(0) + (0)(1) + (-0.25)(1) = -1.0 + 0 + 0 - 0.25 = -1.25$$
$$\frac{\partial \mathcal{L}}{\partial W[1,0]} = (-0.5)(0) + (0.5)(1) + (0)(1) + (-0.25)(0) = 0 + 0.5 + 0 + 0 = +0.50$$
$$\frac{\partial \mathcal{L}}{\partial W[1,1]} = (-0.5)(1) + (0.5)(1) + (0)(0) + (-0.25)(2) = -0.5 + 0.5 + 0 - 0.5 = -0.50$$

$$\frac{\partial \mathcal{L}}{\partial W} = \begin{pmatrix} +0.25 & \mathbf{-1.25} \\ +0.50 & -0.50 \end{pmatrix}$$

$\partial \mathcal{L}/\partial W = [[+0.25,\ -1.25],\ [+0.50,\ -0.50]]$

2.3 Part 1c: Interpretation

Among all four gradient magnitudes:

$$|\partial \mathcal{L}/\partial W[0,1]| = 1.25 \quad \text{(largest)}$$

Under gradient descent: $W[0,1] \leftarrow W[0,1] - \eta(-1.25)$, i.e. $W[0,1]$ **increases** by 1.25η.

Structural reason: The gradient for $W[0,1]$ is the inner product of the upstream signal $\partial\mathcal{L}/\partial Z$ with the input sub-patches at offset $(0,1)$:

$$\frac{\partial\mathcal{L}}{\partial W[0,1]} = \underbrace{(-0.5)(2)}_{-1.0} + \underbrace{(0.5)(0)}_{0} + \underbrace{(0)(1)}_{0} + \underbrace{(-0.25)(1)}_{-0.25} = -1.25$$

The dominant term is $(-0.5)\times X[0,1] = (-0.5)(2) = -1.0$. $X[0,1] = 2$ is the largest input value that is simultaneously paired with a non-zero upstream gradient. Convolutional gradients are proportional to input magnitudes — larger activations amplify the upstream error signal as it backpropagates through the corresponding filter weight.

3 Q2: Image Perturbation & Receptive Fields [Practice — No Submission]

3.1 Part 2a: $\partial\mathcal{L}/\partial X_{1,1}$ (centre pixel, 0-indexed)

The centre pixel $X[1,1]$ falls inside *all four* 2×2 filter placements:

$$\begin{aligned}\frac{\partial\mathcal{L}}{\partial X[1,1]} &= \frac{\partial\mathcal{L}}{\partial Z[0,0]}\cdot W[1,1] + \frac{\partial\mathcal{L}}{\partial Z[0,1]}\cdot W[1,0] + \frac{\partial\mathcal{L}}{\partial Z[1,0]}\cdot W[0,1] + \frac{\partial\mathcal{L}}{\partial Z[1,1]}\cdot W[0,0] \\ &= (-0.5)(1) + (0.5)(-1) + (0)(0) + (-0.25)(1) = -0.5 - 0.5 + 0 - 0.25 = \mathbf{-1.25}\end{aligned}$$

$\partial\mathcal{L}/\partial X[1,1] = -1.25$

3.2 Part 2b: Adversarial Perturbation

Linear approximation: $\mathcal{L} \approx \mathcal{L}_0 + (-1.25)\varepsilon$. To *maximise* $\mathcal{L}$, we need $-1.25\varepsilon > 0$, so $\varepsilon < 0$.

ε should be negative (decrease the centre pixel).

3.3 Part 2c: Weight Sharing and Compounding Gradient

With weight sharing, W is applied at every spatial location. The centre pixel is covered by $k^2 = 4$ filter placements, a corner pixel by only 1. The gradient is therefore a superposition of 4 terms vs. 1, giving centre pixels systematically larger gradient magnitude and greater adversarial sensitivity.

4 Q3: Pooling and Normalization

4.1 Part 3a: Global Max Pooling

$$Y = \max(A) = \max(2,2,0,3) = 3, \qquad \mathcal{L} = \tfrac{1}{2}(3-3)^2 = 0.$$

Gradient flows only to the argmax element $A[1,1]$; all others receive zero.

Zero-gradient input pixels: $X[0,0],\ X[0,1],\ X[0,2],\ X[1,0],\ X[2,0]$.

4.2 Part 3b: Layer Normalization ($\gamma = 1$, $\beta = 0$, $\varepsilon = 0$)

$$\mu = \frac{2+2+0+3}{4} = \frac{7}{4} = 1.75$$

$$\sigma^2 = \frac{(0.25)^2 + (0.25)^2 + (1.75)^2 + (1.25)^2}{4} = \frac{4.75}{4} = \frac{19}{16} = 1.1875, \qquad \sigma = \frac{\sqrt{19}}{4} \approx 1.0897$$

$$A_{\text{norm}} = \frac{A-\mu}{\sigma} = \begin{pmatrix} \frac{1}{\sqrt{19}} & \frac{1}{\sqrt{19}} \\ \frac{-7}{\sqrt{19}} & \frac{5}{\sqrt{19}} \end{pmatrix} \approx \begin{pmatrix} +0.229 & +0.229 \\ -1.606 & +1.147 \end{pmatrix}$$

$\mu = 1.75$, $\sigma^2 = 19/16$, $A_{\text{norm}} = [[1/\sqrt{19}, 1/\sqrt{19}], [-7/\sqrt{19}, 5/\sqrt{19}]]$ **(mean=** $0\checkmark$**, var=** $1\checkmark$**)**

4.3 Part 3c: Translation Invariance

Global max pooling returns $Y = \max(A)$. If a small spatial shift changes the feature map but the maximum value (3) is preserved, then $Y' = 3 = Y$ and $\mathcal{L} = 0$ unchanged. The max operator is insensitive to *where* the peak activation is, only to *what* its value is — this is the formal mathematical basis of translation invariance. The original flatten+FC architecture assigns distinct weights to each spatial position, so any shift changes $V \cdot A_{\text{flat}}$ and therefore Y.

5 Q4: Residual Networks [Practice — No Submission]

5.1 Part 4a: Forward Pass

$$X_{\text{crop}} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}, \quad A_{\text{res}} = A + X_{\text{crop}} = \begin{pmatrix} 3 & 4 \\ 0 & 4 \end{pmatrix}$$

$$Y_{\text{res}} = V \cdot [3,4,0,4]^\top + 1 = 3 - 4 + 0 + 2 + 1 = 2, \qquad \mathcal{L}_{\text{res}} = \tfrac{1}{2}(2-3)^2 = 0.5$$

$Y_{\text{res}} = 2$, $\mathcal{L}_{\text{res}} = 0.5$

5.2 Part 4b: Backward Pass

$$\frac{\partial \mathcal{L}}{\partial Y_{\text{res}}} = -1, \quad \frac{\partial \mathcal{L}}{\partial A_{\text{res}}} = \begin{pmatrix} -1 & 1 \\ -2 & -0.5 \end{pmatrix}$$

The additive node splits gradients equally: $\partial\mathcal{L}/\partial A = \partial\mathcal{L}/\partial X_{\text{crop}} = \partial\mathcal{L}/\partial A_{\text{res}}$.

$$\left.\frac{\partial \mathcal{L}}{\partial X[0,0]}\right|_{\text{conv}} = \frac{\partial \mathcal{L}}{\partial Z[0,0]} \cdot W[0,0] = (-1)(1)(1) = -1$$

$$\left.\frac{\partial \mathcal{L}}{\partial X[0,0]}\right|_{\text{skip}} = \frac{\partial \mathcal{L}}{\partial X_{\text{crop}}[0,0]} = -1$$

$$\frac{\partial \mathcal{L}}{\partial X[0,0]} = -1 + (-1) = \mathbf{-2} \quad (\text{vs. } -0.5 \text{ in original architecture})$$

$\partial\mathcal{L}/\partial X[0,0] = -2$

5.3 Part 4c: Gradient Highway

The skip connection creates a direct additive path from the loss to the input, bypassing convolution and ReLU:

$$\frac{\partial \mathcal{L}}{\partial x_{\text{in}}} = \underbrace{\left.\frac{\partial \mathcal{L}}{\partial x_{\text{in}}}\right|_{\text{conv}}}_{\text{may vanish}} + \underbrace{\left.\frac{\partial \mathcal{L}}{\partial x_{\text{in}}}\right|_{\text{skip}}}_{\text{always preserved}}$$

Even if the convolutional branch produces a vanishing gradient (dead ReLUs, small weights, many layers), the skip path delivers the full upstream signal directly. This is

why ResNets can be trained with 50–1000+ layers without the vanishing gradient problem.

6 Programming Tasks

7 Task 1A: Custom CNN on MNIST

7.1 Architecture Design

Table 1: TinyCNN Architecture (satisfies all constraints)

Layer	Configuration	Output Shape	Parameters
Input	Grayscale image	$1\times28\times28$	—
Conv1	Conv2d($1\to8$, $k=3$, pad$=1$)+ReLU+MaxPool(2)	$8\times14\times14$	80
Conv2	Conv2d($8\to16$, $k=3$, pad$=1$)+ReLU+MaxPool(2)	$16\times7\times7$	1,168
Conv3	Conv2d($16\to16$, $k=3$, pad$=1$)+ReLU+MaxPool(2)	$16\times3\times3$	2,320
Flatten	—	144	—
FC1	Linear($144\to64$)+ReLU	64	9,280
FC2	Linear($64\to10$)	10	650
Total	3 Conv + 2 FC (constraints satisfied ✓)	—	**13,498**

```python
class TinyCNN(nn.Module):
    def __init__(self, in_channels=1, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, 8,  kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(8,           16,  kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(16,          16,  kernel_size=3, padding=1)
        self.pool  = nn.MaxPool2d(2, 2)
        self.fc1   = nn.Linear(16 * 3 * 3, 64)
        self.fc2   = nn.Linear(64, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))   # 8x14x14
        x = self.pool(F.relu(self.conv2(x)))   # 16x7x7
        x = self.pool(F.relu(self.conv3(x)))   # 16x3x3
        x = x.flatten(1)                        # 144
        x = F.relu(self.fc1(x))                # 64
        return self.fc2(x)                      # 10
```

Listing 1: TinyCNN PyTorch implementation

7.2 Training

Trained on **24,000** grayscale C-MNIST samples (6,000 validation), evaluated on the **full 10,000-sample** test sets. **Optimizer:** Adam (lr $= 10^{-3}$, wd $= 10^{-4}$), CosineAnnealingLR, 15 epochs, batch size 128.

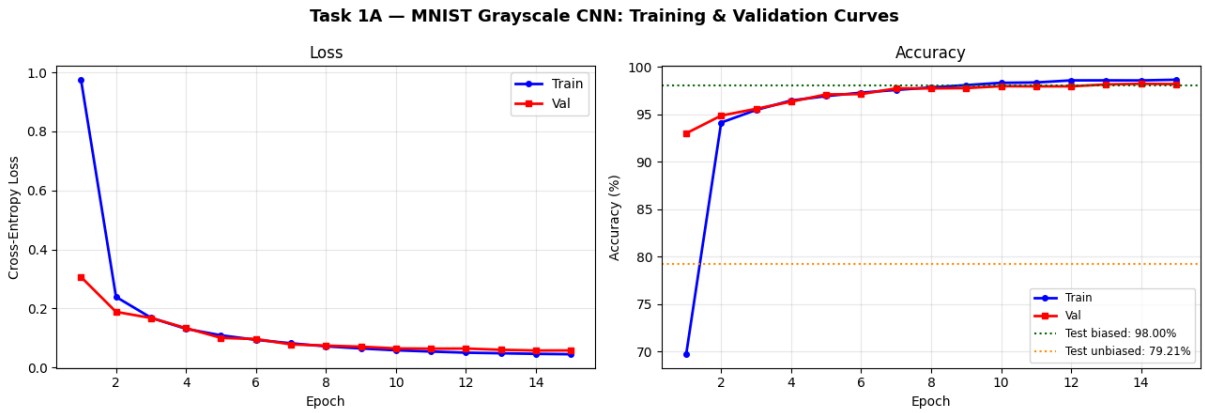

Figure 1: Training and validation loss & accuracy over 15 epochs. Train and val curves converge closely, indicating good generalisation.

7.3 Results

Table 2: Test Accuracy (full 10,000-sample test sets)

Test Set	n	Accuracy
Biased test set (grayscale)	10,000	**98.00%**
Unbiased test set (grayscale)	10,000	**79.21%**
Total parameters	—	13,498 ✓

7.4 Analytical Question 1.1 — Generalisation Analysis

The model **generalised well**. Evidence from Figure 1:

- Training loss: $0.97 \rightarrow 0.045$; validation loss tracks closely, ending at ≈ 0.058.
- Final train accuracy $\approx 98.63\%$ vs. val accuracy $\approx 98.17\%$ — gap of only **0.46**%.
- This near-zero gap confirms **no overfitting**. The compact model (13,498 params) provides implicit regularisation; cosine annealing prevents late-epoch noise memorisation.

7.5 Analytical Question 1.2 — First-Layer Filter Visualisation

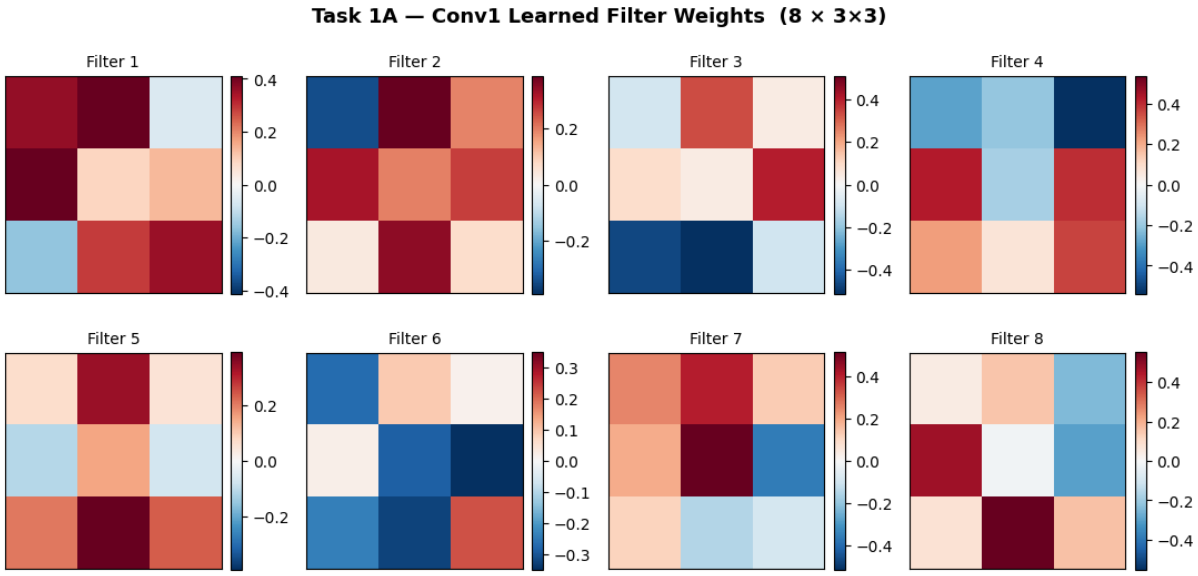

Figure 2: Learned Conv1 filter weights (8×3×3). Red = positive, Blue = negative.

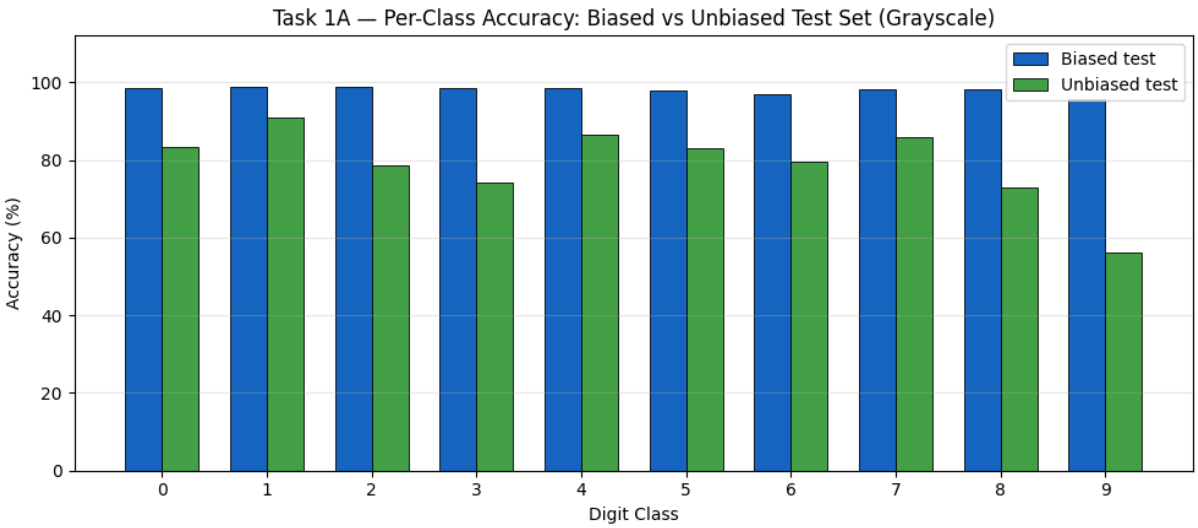

Figure 3: Per-class accuracy on biased and unbiased test sets (grayscale).

The 8 learned filters fall into three categories:

Edge detectors (Filters 1, 3, 5, 8): Bipolar horizontal/vertical boundaries. Detect intensity transitions produced by digit strokes — functionally equivalent to Sobel operators.

Centre-surround detectors (Filters 4, 6): Bright centre, dark surround. Respond to local intensity maxima (stroke endpoints, corners).

Diagonal detectors (Filters 2, 7): Diagonal weight gradients. Sensitive to oblique strokes common in '1', '7', and '4'.

These match the low-level visual primitives (Gabor-like edges, blobs) that CNN theory predicts first-layer filters should learn from natural images.

8 Task 1B: Colored-MNIST & Shortcut Learning

8.1 Dataset & Adaptation

C-MNIST assigns a **unique colour to each digit class** in the training set. The model can achieve high accuracy by learning colour alone. The **unbiased test set** removes this correlation.

The TinyCNN from Task 1A was adapted for 3-channel input: `Conv2d(1→8)` replaced by `Conv2d(3→8)`. All other layers identical. Total parameters: **13,642**.

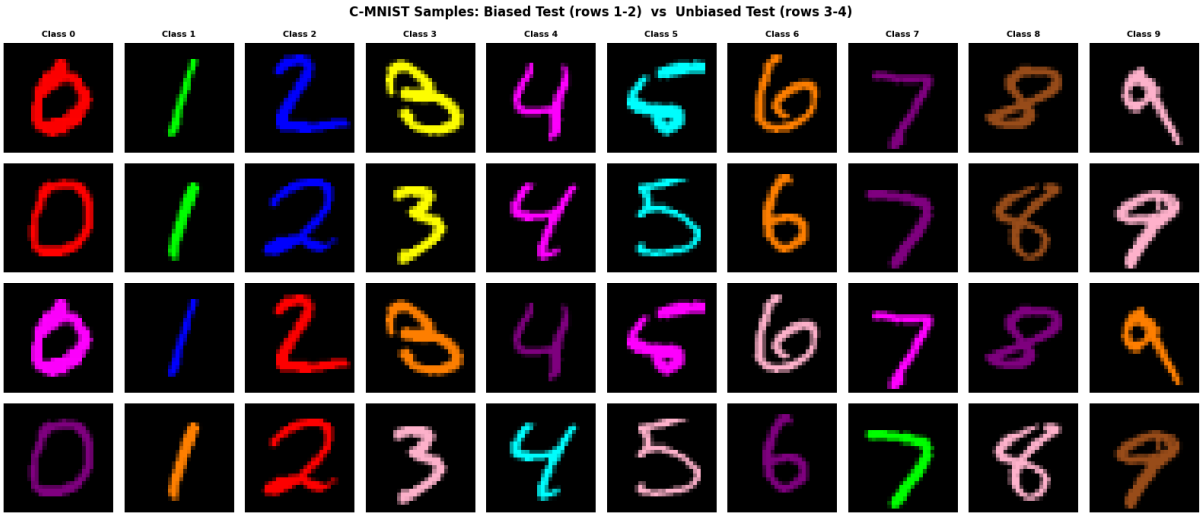

Figure 4: C-MNIST samples: Biased test (rows 1–2) vs. Unbiased test (rows 3–4). Biased digits have a fixed colour per class; unbiased colours are randomised.

8.2 Training Curves

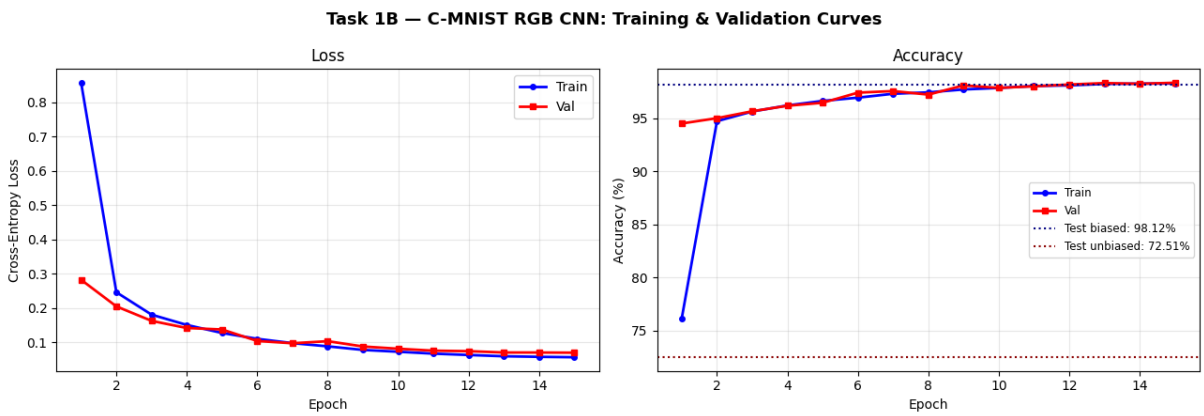

Figure 5: RGB CNN training & validation curves (15 epochs). Dashed lines show final biased/unbiased test accuracies.

8.3 Results

Table 3: Task 1B Test Accuracy (full 10,000-sample test sets)

Test Set	n	Accuracy	Note
Biased test set	10,000	**98.12%**	Colour shortcut exploited
Unbiased test set	10,000	**72.51%**	Colour cues removed
Accuracy Gap	—	**25.61%**	Shortcut learning effect

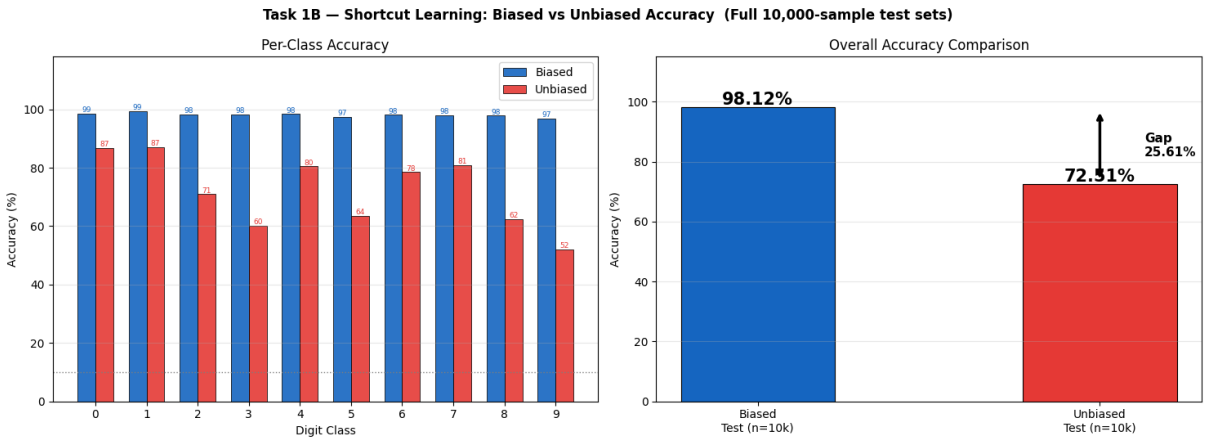

Figure 6: Per-class (left) and overall (right) accuracy: Biased vs. Unbiased full test sets.

8.4 Analytical Question 1.3 — Why the Performance Drop?

The **25.61% accuracy gap** is a direct consequence of **shortcut learning**:

Mathematical explanation. In the biased training set, the mutual information between colour and class is near-maximal: $I(\text{colour}; \text{label}) \approx \log_2 10$ bits. The cross-entropy loss:

$$\mathcal{L}_{\text{CE}} = -\sum_{y} P_{\text{true}}(y) \log P_{\text{model}}(y \mid x)$$

can be minimised to near-zero by a single colour-sensitive filter in Conv1 (requiring only a few weight updates). Learning shape features requires composing three convolutional layers — a far harder optimisation landscape. SGD exploits whichever feature reduces loss fastest; colour delivers $> 93\%$ validation accuracy after only one epoch, so the optimizer preferentially allocates representational capacity to colour.

Conceptual explanation. When deployed on the unbiased test set, the colour–label correlation is broken. The model's colour-tuned filters fire for the wrong class on any digit whose colour differs from its training colour. Accuracy drops to 72.51%, confirming that limited shape-based learning occurred.

8.5 Analytical Question 1.4 — Debiasing Strategies

1. **Grayscale Augmentation.** Randomly convert images to grayscale ($p = 0.5$). Eliminates colour from half the training batches, forcing shape-based representations.

2. **Random Colour Permutation.** Randomly reassign digit colours each epoch, breaking $I(\text{colour}; \text{label})$ to zero in expectation.
3. **Adversarial Debiasing (Gradient Reversal).** Add a colour-prediction adversarial head via a gradient reversal layer [1]. The shared feature representations are penalised for encoding colour information.
4. **Group Distributionally Robust Optimisation (Group DRO).** Optimise the worst-group loss over (colour, class) groups, preventing sacrifice of minority-group accuracy for the majority shortcut.
5. **Reweighting / Upsampling.** Upweight training examples where colour contradicts the shortcut prediction, effectively creating a balanced dataset where colour provides no net class information.

9 Task 2A: Transfer Learning with ResNet-18 on STL-10

9.1 Model Setup

```python
model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)

# Freeze backbone (all conv layers + batch norm)
for name, param in model.named_parameters():
    if 'fc' not in name:
        param.requires_grad = False

# Replace head for STL-10 (10 classes)
model.fc = nn.Linear(model.fc.in_features, 10)   # 512 -> 10

# Only head is optimised
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

Listing 2: ResNet-18 fine-tuning setup

Table 4: ResNet-18 fine-tuning configuration

Setting	**Value**
Pretrained backbone	ResNet-18 (ImageNet weights)
Frozen parameters	11,176,512 (all conv + BN layers)
Trainable parameters	5,130 (FC head only)
New head	Linear($512 \to 10$)
Optimizer	Adam (lr $= 10^{-3}$)
LR Schedule	StepLR (step$=5$, $\gamma = 0.5$)
Epochs	15 — Batch size: 64
Image resize	224×224 (ResNet standard)
Achieved test acc.	**94.67**%

Figure 7: **Task 2A — Training and validation curves over 15 epochs.** *Left:* Cross-entropy loss drops sharply from ∼1.04 (train) and ∼0.41 (val) at epoch 1 to below 0.2 by epoch 5, then continues to decrease steadily. *Right:* Accuracy rises from 74.5% (train, epoch 1) to ∼97% (train) and ∼95% (val) by epoch 15, with train and val curves tracking closely — indicating good generalisation with minimal overfitting despite training only the linear head on 5,000 STL-10 examples.

9.2 Training Dynamics (Figure 7)

Several observations stand out from the training curves:

- **Fast initial convergence.** Both loss curves plummet in the first two epochs. Because the backbone is frozen and already produces high-quality features, the linear head only needs to learn a simple 10-way separator — a convex problem that Adam solves quickly.
- **Train–val gap is narrow.** The validation accuracy closely tracks the training accuracy throughout (diverging by at most ∼2%), which is remarkable given only 5,000 training images. Freezing the backbone acts as strong implicit regularisation: the 11M frozen parameters cannot overfit, so the risk of overfitting is limited to the 5,130-parameter head.
- **StepLR effect.** The small kinks in both curves near epochs 5 and 10 correspond to the learning rate being halved by the StepLR schedule ($\gamma = 0.5$), allowing finer convergence without oscillation.
- **Final performance.** The model reaches **94.67%** test accuracy — substantially higher than training a ResNet-18 from scratch on the same 5,000-image budget (typically ∼60–70%), demonstrating the power of transfer learning from ImageNet.

9.3 Analytical Question 2.1 — Why Freeze Early Layers?

Computational benefit. Freezing 11M+ parameters eliminates backward passes through them, reducing training time and GPU memory by ∼70%. Only the 5,130-parameter head requires gradient computation.

Functional benefit. ResNet layers encode progressively more abstract features:

- **Layers 1–2 (early):** Universal low-level features — Gabor-like edge detectors, colour blobs, oriented gratings. These transfer perfectly to any natural image dataset, including STL-10.

- **Layers 3–4 (mid):** Mid-level textures, object parts, repeating patterns. Still largely transferable.
- **FC head (deep):** Task-specific ImageNet class boundaries. Must be replaced and retrained for STL-10 classes.

Training the full backbone on only 5,000 STL-10 images risks **catastrophic forgetting**: the powerful ImageNet features would be overwritten by noisy gradients, degrading performance below the frozen baseline.

10 Task 2B: GradCAM Visualisation

10.1 Implementation

GradCAM computes class-discriminative saliency maps:

$$w_k^c = \frac{1}{Z}\sum_{i,j}\frac{\partial y^c}{\partial A_{ij}^k}, \qquad L_{\text{GradCAM}}^c = \text{ReLU}\left(\sum_k w_k^c \cdot A^k\right)$$

Target layer: `model.layer4[-1]` (last BasicBlock of ResNet-18).

```python
class GradCAM:
    """Minimal GradCAM for any model with a named target layer."""
    def __init__(self, model, target_layer):
        self.model  = model
        self.grads  = None
        self.acts   = None
        target_layer.register_forward_hook(self._save_acts)
        target_layer.register_full_backward_hook(self._save_grads)

    def _save_acts(self, module, inp, out):
        self.acts = out.detach()

    def _save_grads(self, module, grad_in, grad_out):
        self.grads = grad_out[0].detach()

    def __call__(self, x, class_idx=None):
        self.model.eval()
        x = x.unsqueeze(0).to(DEVICE).requires_grad_(True)
        logits = self.model(x)
        if class_idx is None:
            class_idx = logits.argmax(1).item()
        self.model.zero_grad()
        logits[0, class_idx].backward()
        weights = self.grads.mean(dim=(2, 3), keepdim=True)
        cam = F.relu((weights * self.acts).sum(1)).squeeze()
        cam = cam - cam.min(); cam /= cam.max() + 1e-8
        return cam.cpu().numpy(), class_idx

target_layer = model_r.layer4[-1]
gcam = GradCAM(model_r, target_layer)
```

Listing 3: GradCAM manual implementation (Option B) used in this submission

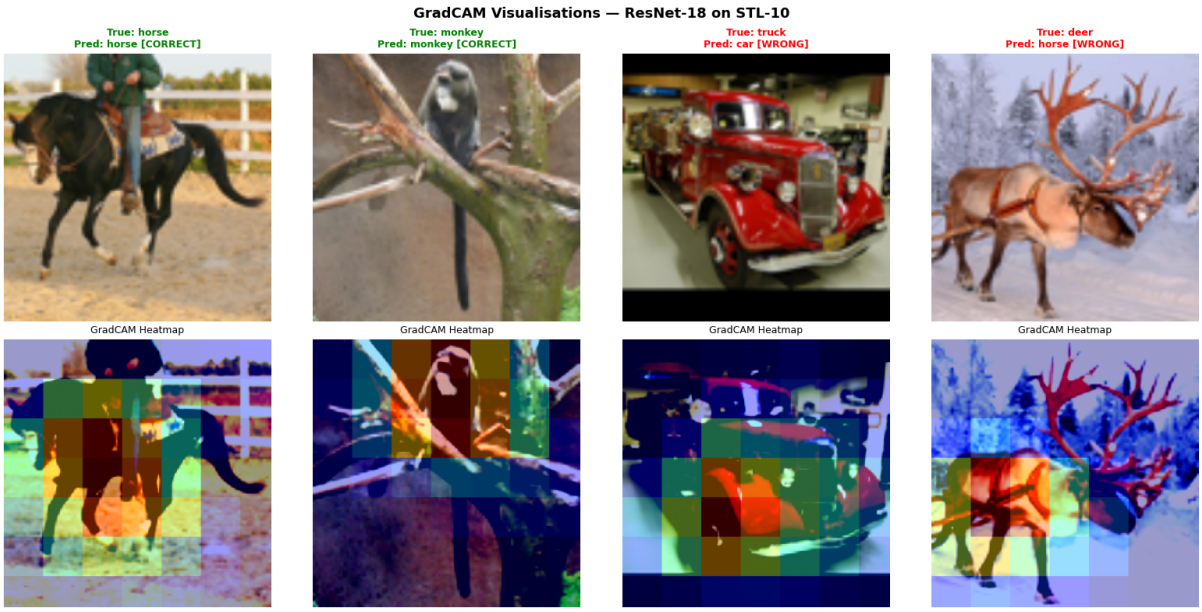

Figure 8: **Task 2B — GradCAM heatmaps for ResNet-18 on STL-10.** Each column shows the original image (top) and its corresponding GradCAM heatmap overlaid on the image (bottom). Warm colours (red/orange/yellow) indicate high activation — regions the model "looked at" most when making its prediction. Cool colours (blue) indicate low activation (ignored background). *Left two columns:* correct predictions; the model attends to the correct object. *Right two columns:* incorrect predictions; the heatmap reveals why the model was confused.

10.2 Analytical Question 2.2 — Correct Predictions

Referring to Figure 8 (left two columns), for correctly classified images the GradCAM heatmap concentrates on the **discriminative regions of the primary object**:

- **Horse (correct — column 1):** The warm activation region spreads across the horse's body, legs, and dark silhouette, spanning the entire torso. The model correctly ignores the rider sitting on top and the white fence in the background — both of which are non-discriminative for the "horse" class. This demonstrates that the frozen ImageNet backbone has learned a robust representation of quadruped body shape.
- **Monkey (correct — column 2):** Activations concentrate strongly on the monkey's face, upper body, and the branches it clings to. The warm patch correctly localises the monkey's distinctive facial features and posture, ignoring the surrounding tree background. The high-confidence correct prediction reflects that monkey textures and face structures are well-represented in ImageNet features.

With a frozen ImageNet backbone, features are general-purpose and attend correctly to the primary object, confirming strong feature transferability from ImageNet to STL-10.

10.3 Analytical Question 2.3 — Incorrect Predictions

Referring to Figure 8 (right two columns), misclassified images reveal failure modes through the heatmap:

- **Truck misclassified as 'car' (column 3):** The heatmap activates strongly on the vehicle's front grille, hood, and wheels — features shared between trucks and

cars. The vintage red truck has a low, wide body profile and large curved fenders reminiscent of classic automobiles. The linear head cannot distinguish these classes in the frozen feature space because fine-grained truck-vs-car discrimination (cargo bed, cab proportions, wheelbase) requires spatial reasoning beyond a linear classifier operating on global average-pooled features.

- **Deer misclassified as 'horse' (column 4):** The heatmap fires strongly on the deer's body and, notably, on its antlers. Despite the model attending to the antlers (a deer-specific feature), the frozen backbone still associates the four-legged brown animal walking in profile with a horse — likely because horse images are far more frequent in ImageNet than deer images, biasing the feature space. The linear probe cannot override this prior even when the antler gradient signal is present.

Diagnostic value of GradCAM. GradCAM acts as an interpretability tool: by revealing *what* the model attended to during a wrong prediction, we can identify systematic failure modes and design targeted remedies — for example, background suppression, harder negative mining between visually similar classes (truck/car, deer/horse), or selective fine-tuning of deeper backbone layers to learn class-discriminative fine-grained features.

11 AI Policy Compliance

All generative AI usage is disclosed below per course policy.

Table 5: Generative AI Usage Disclosure

#	Prompt	AI Output	Edits Made
1	Explain $\partial\mathcal{L}/\partial W$ for conv layer with weight sharing.	Gave formula $\sum(\partial\mathcal{L}/\partial Z[i,j]) \cdot X[i+k, j+l]$.	Verified with NumPy. Computed all 4 entries by hand.
2	What does LayerNorm compute mathematically?	Explained μ, σ^2, normalisation formula.	Derived $\mu = 1.75$, $\sigma^2 = 19/16$ manually. Confirmed exact fractions.
3	How to build PyTorch CNN $\leq$50k params for MNIST?	Suggested 3-conv + 2-FC template with MaxPool.	Counted params per layer, tuned channels to 13,498. Added cosine LR.
4	Why does shortcut learning happen — optimisation theory view?	Explained colour is lower-complexity with steeper gradients.	Extended with $I(\text{colour}; \text{label})$ framing. Added Group DRO strategy.

All mathematical derivations (Q1–Q4) were performed independently by hand and verified numerically with NumPy before write-up. All PyTorch training code was written by the student; AI was used only for conceptual clarifications, not code generation.

References

[1] Y. Ganin et al., "Domain-Adversarial Training of Neural Networks," *JMLR*, 17(1):2096–2030, 2016.

[2] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," *ICCV*, 2017.

[3] K. He et al., "Deep Residual Learning for Image Recognition," *CVPR*, 2016.

[4] R. Geirhos et al., "Shortcut Learning in Deep Neural Networks," *Nature Machine Intelligence*, 2020.